Rotten code, aging standards, & pwning IPv4 parsing across nearly every mainstream programming language


EVERYONE, PLEASE STAY CALM
WE ARE RELEASING POC'S FOR
THE UNPATCHED RCE SO YOU CAN
BETTER EVALUATE YOUR SECURITY POSTURE!
imgflip.com

@jfslowik

Disclaimer

- None of this research was paid for
- We research in good faith
- Nothing today represents our past/present/future employers
- None of us are under gag orders*
- All images are CC0 or public domain
- All trademarks, logos and brand names are the property of their respective owners

What will we be discussing today?

- **CVE-2020-28360** **9.8**
- **CVE-2021-28918** **9.1**
- **CVE-2021-29418** **5.3** (@Ryotkak's emergency fix)
- **CVE-2021-29921** **9.8**
- **CVE-2021-29662** **7.5**
- **CVE-2021-29424** **7.5**
- **CVE-2021-29922** **new** est. 9.8
- **CVE-2021-29923** **new** est. 9.8
- Oracle S1446698 **pending** est. 9.8
- ***** 768013610 **pending** est. 9.8

Talk format

- Finding this type of vuln
- How to horizontally scale your vulnerabilities
- Some PoCs
- Exploitability of a vuln
- Further attack vectors/research

Takeaways

- #patchtuesday every day
- Patching* an entire class of this size?

**Apologies to anyone who might have to go patch right after this*

Takeaways

- Exponential vulnerability disclosures
- Thought models to magnify your attack vectors
- Horizontally scale your research

What do you see?

Severity

CVSS Version 3.x | CVSS Version 2.0

CVSS 3.x Severity and Metrics:

NIST: NVD **Base Score:** 9.8 CRITICAL **Vector:** CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

NVD Analysts use publicly available information to associate vector strings and CVSS scores. We also display any CVSS information provided within the CVE List from the CNA.

Note: NVD Analysts have published a CVSS score for this CVE based on publicly available information at the time of analysis. The CNA has not provided a score within the CVE List.

Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

Severity

CVSS V

CVSS 3.x Severity and P

CVSS scores. We d ***he***

NIST: NVD

U/C:H/I:H/A:H

vailable informat ***d***

NVD Analysts use publicly ava ***vided within the CVE List from the CNA.***

Note: NVD Analysts have publ ***as not provided a score within the CVE List.***

CVSS v3.1 Severity and Metrics:

Base Score: 9.8 CRITICAL

Vector: AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

Impact Score: 5.9

Exploitability Score: 3.9

Attack Vector (AV): Network

Attack Complexity (AC): Low

Privileges Required (PR): None

User Interaction (UI): None

Scope (S): Unchanged

Confidentiality (C): High

Integrity (I): High

Availability (A): High

and Too

rovided these have

drawn on acco from this

purpose. NIST does not necessarily endorse the views expressed

You're listening to...

Sick Codes **@sickcodes**

Kelly Kaoudis **@kaoudis**

Presenting our work with...

John Jackson **@johnjhacking**

Nick Sahler **@tensor_bodega**

Victor Viale **@Koroeskohr**

Cheng Xu **@_xucheng_**

Harold Hunt **@huntharo**

Quickstart

Octal (base-8) number system

0 1 2 3 4 5 6 7

https://linguistics.berkeley.edu/~avelino/Avelino_2006.pdf

No 8s or 9s, 0–7 only

0o100

1 * 8^2 = 64
0 * 8^1 = 0
0 * 8^0 = 0

64

Leaving home…

```
0o127
```

```
1 * 8^2 = 64
2 * 8^1 = 16
7 * 8^0 = 7
-----------
          87
```

Coming home…

127

```
1 * 10^2 = 100
2 * 10^1 = 20
7 * 10^0 = 7
-----------
        127
```

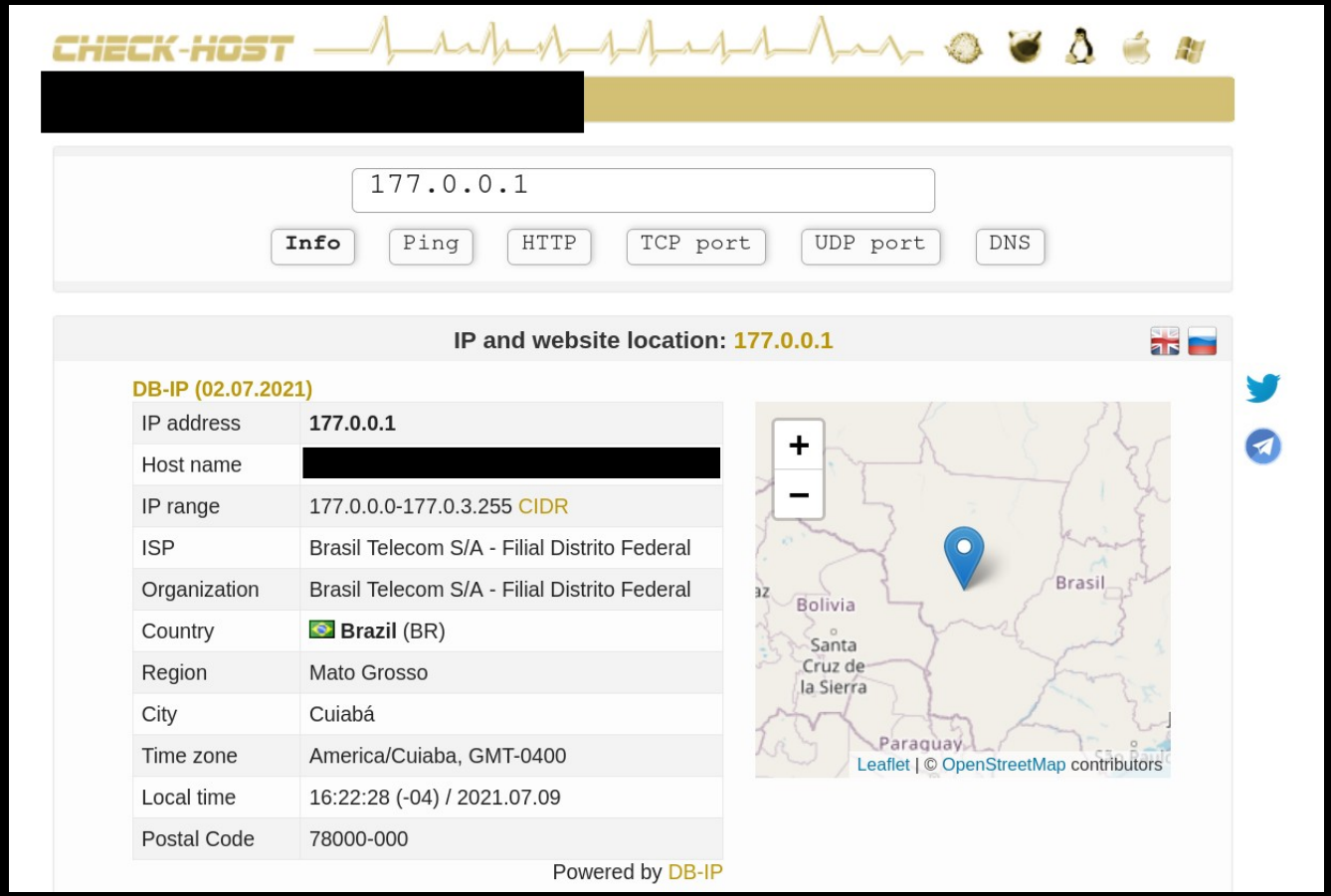
Coming home…

127

```
127/8 = 15   r1
15/8  = 1    r7
1/8   = 0    r7
```

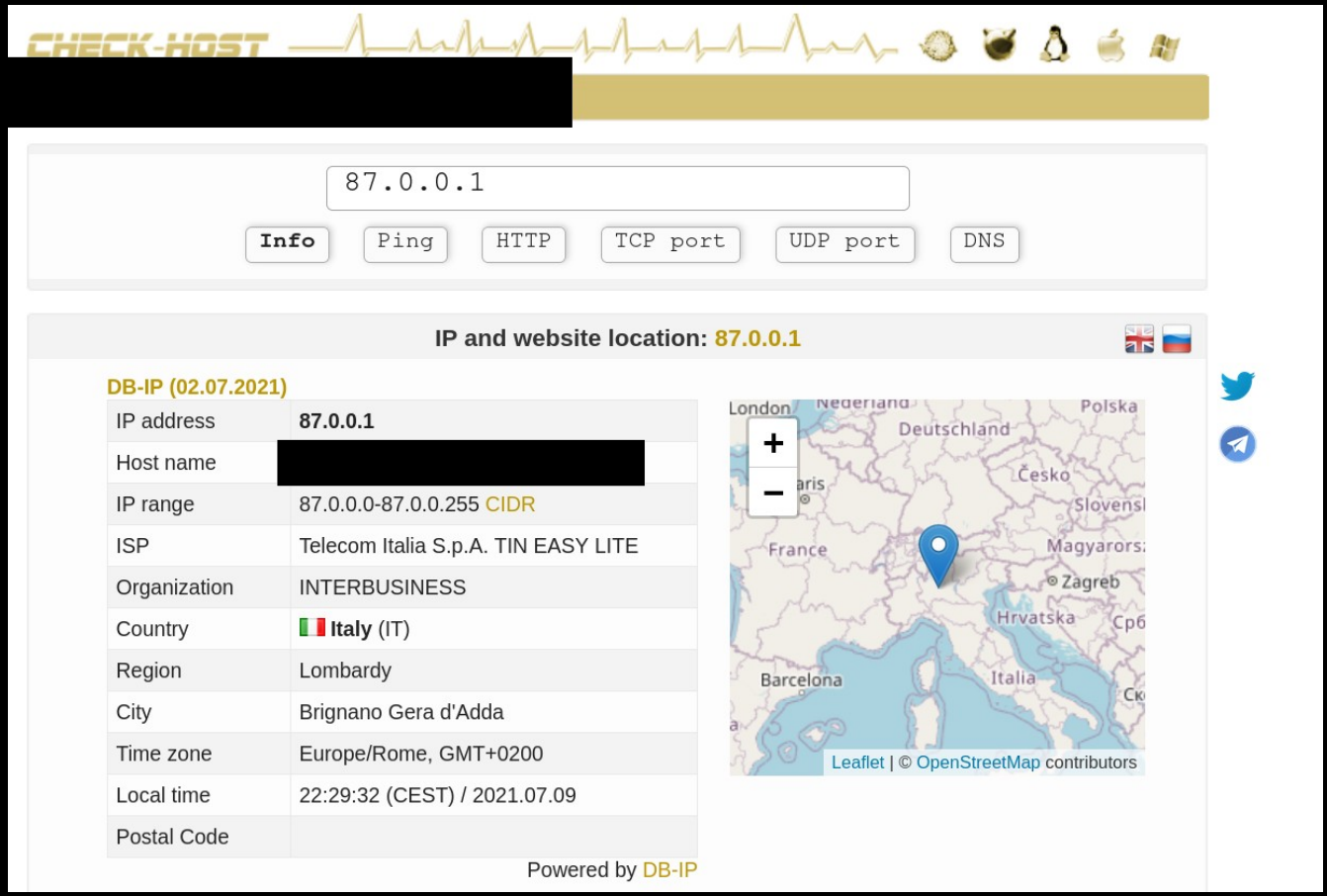
177_8

0177.0.0.1 → 127.0.0.1



https://check-host.net/

87.0.0.1 ← 0127.0.0.1



https://check-host.net/

@bagder (curl guy)

Daniel Stenberg 

@bagder

I learned that getaddrinfo() converts IPv4 addresses given as octal to decimal. See "ping 0177.0.0.1" or even "curl 0177.0.0.1" ... (the latter usually gets a 400 due to the funny Host:)

12:46 PM · Sep 27, 2018 · TweetDeck

@bagder (curl guy)

Daniel Stenberg
@bagder

I learned that getaddrinfo() converts IPv4 addresses given as octal to decimal. See "ping 0177.0.0.1" or even "curl 0177.0.0.1" ... (the latter usually gets a 400 due to the funny Host:)

12:46 PM · Sep 27, 2018

funny Host:)

Rotten code?

- Bitrot, but for dependencies (not blame hour!)
- How well-tested should developers assume a widely-used or standard library is?

Contributing factors

For example, the literal 73 (base 8) might be represented as `073`, `o73`, `q73`, `0o73`, `\73`, `@73`, `&73`, `$73` or `73o` in various languages.

Newer languages have been abandoning the prefix `0`, as decimal numbers are often represented with leading zeroes. The prefix `q` was introduced to avoid the prefix `o` being mistaken for a zero, while the prefix `0o` was introduced to avoid starting a numerical literal with an alphabetic character (like `o` or `q`), since these might cause the literal to be confused with a variable name. The prefix `0o` also follows the model set by the prefix `0x` used for hexadecimal literals in the C language; it is supported by Haskell,[18] OCaml,[19] Python as of version 3.0,[20] Raku,[21] Ruby,[22] Tcl as of version 9,[23] PHP as of version 8.1[24] and it is intended to be supported by ECMAScript 6[25] (the prefix `0` originally stood for base 8 in JavaScript but could cause confusion,[26] therefore it has been discouraged in ECMAScript 3 and dropped in ECMAScript 5[27]).

Contributing factors

For example, the literal 73 (base 8) might be represented as `073`, `o73`, `q73`, `0o73`, `\73`, `@73`, `&73`, `$73` or `73o` in various languages.

Newer languages have been abandoning the prefix `0`, as decimal numbers are often represented with leading zeroes. The prefix `q` was introduced to avoid

therefore it has been discouraged in ECMAScript 3 and dropped in ECMAScript 5[27]).

hexadecimal literals in the C language; it is supported by Haskell,[18] OCaml,[19] Python as of version 3.0,[20] Raku,[21] Ruby,[22] Tcl as of version 9,[23] PHP as of version 8.1[24] and it is intended to be supported by ECMAScript 6[25] (the prefix `0` originally stood for base 8 in JavaScript but could cause confusion,[26] therefore it has been discouraged in ECMAScript 3 and dropped in ECMAScript 5[27]).

Contributing factors

- No (ratified) IP address format standard
- Who even *uses* octal other than hackers anyway

nodejs private-ip

private-ip DT

2.2.1 • Public • Published 2 months ago

Readme | Explore BETA | 2 Dependencies | 14 Dependents | 10 Versions

private-ip

> Check if IP address is private.

Installation

```
❯ npm install private-ip --save

or

❯ yarn add private-ip
```

Install

```
❯ npm i private-ip
```

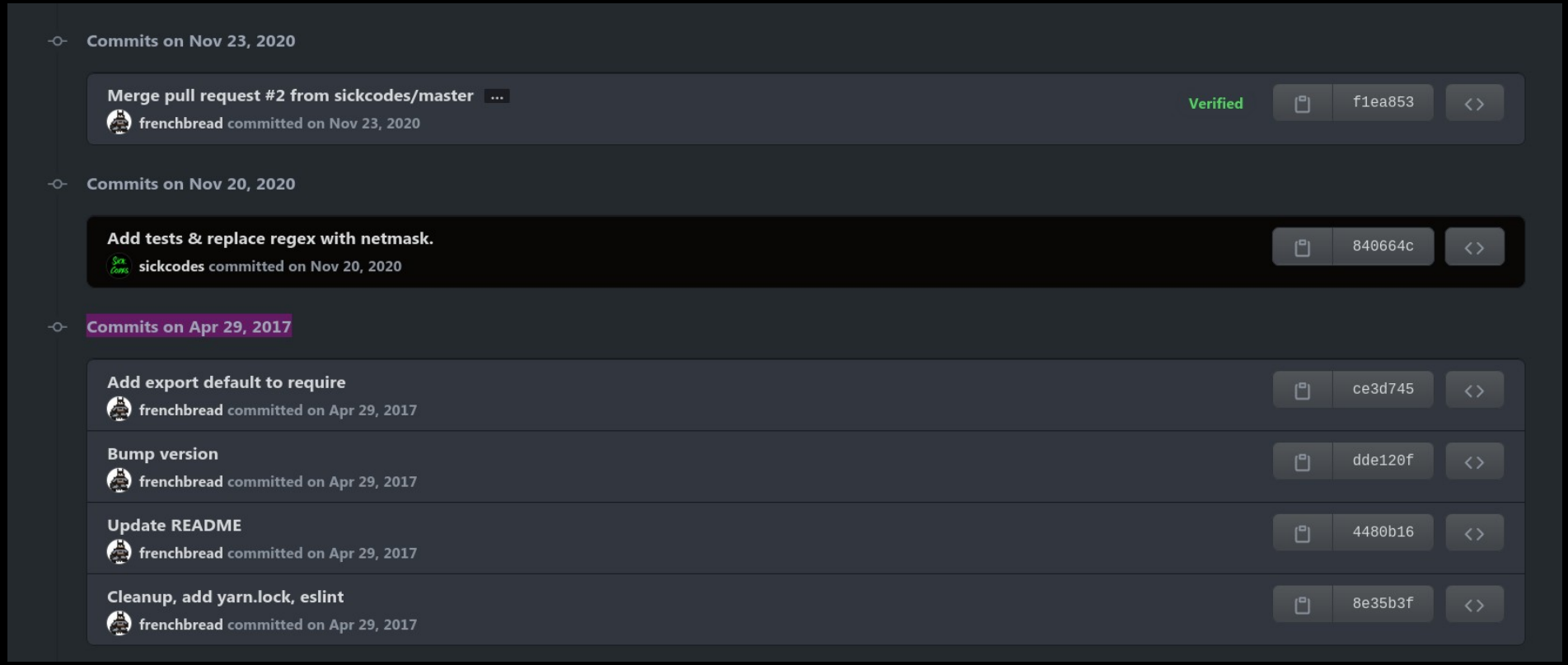
Weekly Downloads

12,184

Version	License
2.2.1	MIT

Unpacked Size	Total Files
9.56 kB	7

private-ip


Commits on Nov 23, 2020
Merge pull request #2 from sickcodes/master ...
frenchbread committed on Nov 23, 2020
Verified
f1ea853
Commits on Nov 20, 2020
Add tests & replace regex with netmask.
sickcodes committed on Nov 20, 2020
840664c
Commits on Apr 29, 2017
Add export default to require
frenchbread committed on Apr 29, 2017
ce3d745
Bump version
frenchbread committed on Apr 29, 2017
dde120f
Update README
frenchbread committed on Apr 29, 2017
4480b16
Cleanup, add yarn.lock, eslint
frenchbread committed on Apr 29, 2017
8e35b3f

```
    34 +
 1  35   export default (ip) => (
 2     -   /^(::f{4}:)?10\.([0-9]{1,3})\.([0-9]{1,3})\.([0-9]{1,3})$/i.test(ip) ||
 3     -   /^(::f{4}:)?192\.168\.([0-9]{1,3})\.([0-9]{1,3})$/i.test(ip) ||
 4     -   /^(::f{4}:)?172\.(1[6-9]|2\d|30|31)\.([0-9]{1,3})\.([0-9]{1,3})$/i.test(ip) ||
 5     -   /^(::f{4}:)?127\.([0-9]{1,3})\.([0-9]{1,3})\.([0-9]{1,3})$/i.test(ip) ||
 6     -   /^(::f{4}:)?169\.254\.([0-9]{1,3})\.([0-9]{1,3})$/i.test(ip) ||
 7     -   /^f[cd][0-9a-f]{2}:/i.test(ip) ||
 8     -   /^fe80:/i.test(ip) ||
 9     -   /^::1$/.test(ip) ||
10     -   /^::$/.test(ip)
    36 +   netmaskCheck(ip)
11  37   )
```

44 src/index.js

```
@@ -1,11 +1,37 @@
+ var Netmask = require('netmask').Netmask
+ function netmaskCheck (params) {
+   let privateRanges = [
+     '0.0.0.0/8',
+     '10.0.0.0/8',
+     '100.64.0.0/10',
+     '127.0.0.0/8',
+     '169.254.0.0/16',
+     '172.16.0.0/12',
+     '192.0.0.0/24',
+     '192.0.0.0/29',
+     '192.0.0.8/32',
+     '192.0.0.9/32',
+     '192.0.0.10/32',
+     '192.0.0.170/32',
+     '192.0.0.171/32',
+     '192.0.2.0/24',
+     '192.31.196.0/24',
+     '192.52.193.0/24',
+     '192.88.99.0/24',
+     '192.168.0.0/16',
+     '192.175.48.0/24',
+     '198.18.0.0/15',
+     '198.51.100.0/24',
+     '203.0.113.0/24',
+     '240.0.0.0/4',
+     '255.255.255.255/32'
+   ].map(b => new Netmask(b))
+   for (let r of privateRanges) {
+     if (r.contains(params)) return true
+   }
+   return false
+ }
```

```
...  @@ -1,11 +1,37 @@
1    + var Netmask = require('netmask').Netmask
2    + function netmaskCheck (params) {
3    +   let privateRanges = [
4    +     '0.0.0.0/8',
5    +     '10.0.0.0/8',
6    +     '100.64.0.0/10',
7    +     '127.0.0.0/8',
8    +     '169.254.0.0/16',
9    +     '172.16.0.0/12',
10   +     '192.0.0.0/24',
```

CVE-2020-28360 Detail

Current Description

Insufficient RegEx in private-ip npm package v1.0.5 and below insufficiently filters reserved IP ranges resulting in indeterminate SSRF. An attacker can perform a large range of requests to ARIN reserved IP ranges, resulting in an indeterminable number of critical attack vectors, allowing remote attackers to request server-side resources or potentially execute arbitrary code through various SSRF techniques.

+View Analysis Description

Severity

CVSS Version 3.x | CVSS Version 2.0

CVSS 3.x Severity and Metrics:

NIST: NVD

Base Score: 9.8 CRITICAL

Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

CVE-2021-28918 Detail

Current Description

Improper input validation of octal strings in netmask npm package v1.0.6 and below allows unauthenticated remote attackers to perform indeterminate SSRF, RFI, and LFI attacks on many of the dependent packages. A remote unauthenticated attacker can bypass packages relying on netmask to filter IPs and reach critical VPN or LAN hosts.

View Analysis Description

Severity

CVSS Version 3.x | CVSS Version 2.0

CVSS 3.x Severity and Metrics:

NVD **NIST:** NVD **Base Score:** 9.1 CRITICAL

Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N

CVE-2021-29418 Detail

Current Description

The netmask package before 2.0.1 for Node.js mishandles certain unexpected characters in an IP address string, such as an octal digit of 9. This (in some situations) allows attackers to bypass access control that is based on IP addresses. NOTE: this issue exists because of an incomplete fix for CVE-2021-28918.

View Analysis Description

Severity

CVSS Version 3.x | CVSS Version 2.0

CVSS 3.x Severity and Metrics:

NVD **NIST:** NVD **Base Score:** 5.3 MEDIUM

Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:L/A:N

"Private Internets"

[Search] [txt|html|pdf|bibtex] [Tracker] [WG] [Email] [Diff1] [Diff2] [Nits]

From: draft-ietf-cidrd-private-addr-05 Best Current Practice
Updated by: 6761 Errata exist
Network Working Group Y. Rekhter
Request for Comments: 1918 Cisco Systems
Obsoletes: 1627, 1597 B. Moskowitz
BCP: 5 Chrysler Corp.
Category: Best Current Practice D. Karrenberg
RIPE NCC
G. J. de Groot
RIPE NCC
E. Lear
Silicon Graphics, Inc.
February 1996

Address Allocation for Private Internets

Status of this Memo

This document specifies an Internet Best Current Practices for the Internet Community, and requests discussion and suggestions for improvements. Distribution of this memo is unlimited.

1. Introduction

For the purposes of this document, an enterprise is an entity autonomously operating a network using TCP/IP and in particular determining the addressing plan and address assignments within that network.

This document describes address allocation for private internets. The

https://datatracker.ietf.org/doc/html/rfc1918

The obvious ranges

RFC 1918 Address Allocation for Private Internets February 1996

3. Private Address Space

The Internet Assigned Numbers Authority (IANA) has reserved the following three blocks of the IP address space for private internets:

10.0.0.0	-	10.255.255.255	(10/8 prefix)
172.16.0.0	-	172.31.255.255	(172.16/12 prefix)
192.168.0.0	-	192.168.255.255	(192.168/16 prefix)

6. Security Considerations

Security issues are not addressed in this memo.

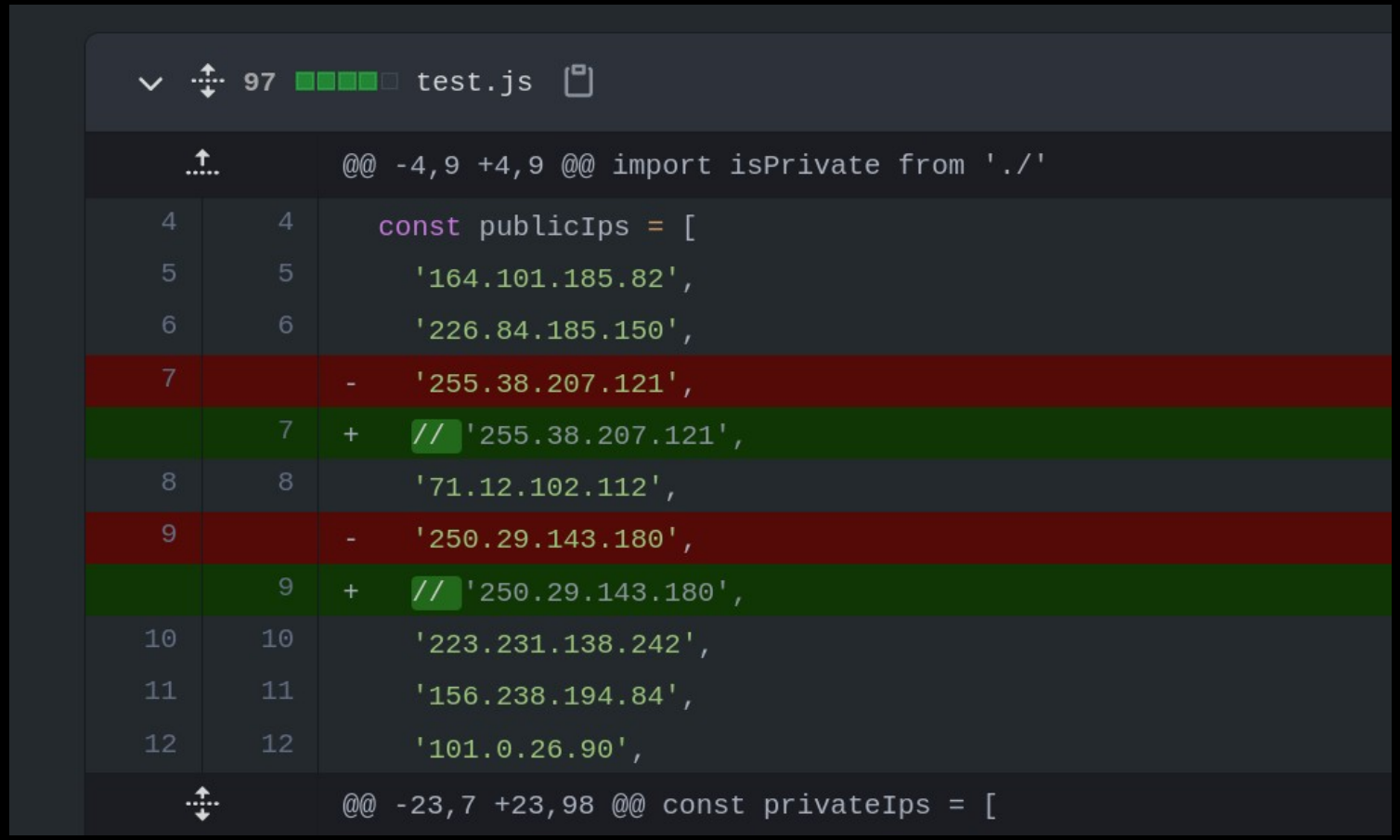
That’ll never happen...

Some examples, where external connectivity might not be required, are:

- A large airport which has its arrival/departure displays individually addressable via TCP/IP. It is very unlikely that these displays need to be directly accessible from other networks.

- Large organizations like banks and retail chains are switching to TCP/IP for their internal communication. Large numbers of local workstations like cash registers, money machines, and equipment at clerical positions rarely need to have such connectivity.

Back to the repo



Some more test cases

```
108 +   '255.192.0.0',
109 +   '255.240.0.0',
110 +   '255.254.0.0',
111 +   '255.255.0.0',
112 +   '255.255.255.0',
113 +   '255.255.255.248',
114 +   '255.255.255.254',
115 +   '255.255.255.255',
116 +   '0000.0000.0000.0000',
117 +   '0000.0000'
27  118   ]
28  119
```

No node-netmask for IPv6

```
- )
+ function ipv6Check (params) {
+   return /^::$/.test(params) ||
+     /^::1$/.test(params) ||
+     /^::f{4}:([0-9]{1,3})\.([0-9]{1,3})\.([0-9]{1,3})\.([0-9]{1,3})$/.test(params) ||
+     /^::f{4}:0.([0-9]{1,3})\.([0-9]{1,3})\.([0-9]{1,3})\.([0-9]{1,3})$/.test(params) ||
+     /^64:ff9b::([0-9]{1,3})\.([0-9]{1,3})\.([0-9]{1,3})\.([0-9]{1,3})$/.test(params) ||
+     /^100::([0-9a-fA-F]{0,4}):?([0-9a-fA-F]{0,4}):?([0-9a-fA-F]{0,4}):?([0-9a-fA-F]{0,4})$/.test(params) ||
+     /^2001::([0-9a-fA-F]{0,4}):?([0-9a-fA-F]{0,4}):?([0-9a-fA-F]{0,4}):?([0-9a-fA-F]{0,4}):?([0-9a-fA-F]{0,4}):?([0-9a-fA-F]{0,4})$/.test(params) ||
+     /^2001:2[0-9a-fA-F]:([0-9a-fA-F]{0,4}):?([0-9a-fA-F]{0,4}):?([0-9a-fA-F]{0,4}):?([0-9a-fA-F]{0,4}):?([0-9a-fA-F]{0,4}):?([0-9a-fA-F]{0,4})$/.test(pa
+     /^2001:db8:([0-9a-fA-F]{0,4}):?([0-9a-fA-F]{0,4}):?([0-9a-fA-F]{0,4}):?([0-9a-fA-F]{0,4}):?([0-9a-fA-F]{0,4}):?([0-9a-fA-F]{0,4})$/.test(params) ||
+     /^2002:([0-9a-fA-F]{0,4}):?([0-9a-fA-F]{0,4}):?([0-9a-fA-F]{0,4}):?([0-9a-fA-F]{0,4}):?([0-9a-fA-F]{0,4}):?([0-9a-fA-F]{0,4}):?([0-9a-fA-F]{0,4})$/
+     /^f[c-d]([0-9a-fA-F]{2,2}):/i.test(params) ||
+     /^fe[8-9a-bA-B][0-9a-fA-F]:/i.test(params) ||
+     /^ff([0-9a-fA-F]{2,2}):/i.test(params)
+ }
+
+ export default (ip) => {
+   if (isIp.v4(ip) || ip.startsWith('0')) {
+     return netmaskCheck(ip)
+   }
+   return ipv6Check(ip)
+ }
```


RegEx

s/regex/netmask/

netmask DT

2.0.2 • Public • Published 3 months ago

Readme | Explore BETA | 0 Dependencies | 169 Dependents | 10 Versions

Netmask

The Netmask class parses and understands IPv4 CIDR blocks so they can be explored and compared. This module is highly inspired by Perl Net::Netmask module.

Synopsis

```
var Netmask = require('netmask').Netmask

var block = new Netmask('10.0.0.0/12');
block.base;                    // 10.0.0.0
block.mask;                    // 255.240.0.0
block.bitmask;                 // 12
block.hostmask;                // 0.15.255.255
block.broadcast;               // 10.15.255.255
block.size;                    // 1048576
```

Install

```
> npm i netmask
```

Weekly Downloads

2,997,509

Version	License
2.0.2	MIT

Unpacked Size	Total Files
32.5 kB	10

Issues	Pull Requests
4	1

CVE-2020-28360

- Harold, John, Nick & Sick

https://johnjhacking.com/blog/cve-2020-28360/

CVE-2020-28360

- **12,120** direct weekly downloads
- **355** publicly identified npm dependents
  (public npm packages relying on private-ip)
- **73** GitHub dependents
  https://github.com/frenchbread/private-ip/network/dependents
- **153,374** combined weekly downloads of all dependents
  (most frequently libp2p-related)
  https://github.com/frenchbread/private-ip/network/dependents

yeah,
so?

@agarri_fr

https://www.agarri.fr/docs/AppSecEU15-Server_side_browsing_considered_harmful.pdf

ssrf

- Bypass input validation / waf rules
- Masquerade as the server to destination
- LFI
- RFI

https://cheatsheetseries.owasp.org/assets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet_SSRF_Bible.pdf

CVE-2021-28918, CVE-2021-29418

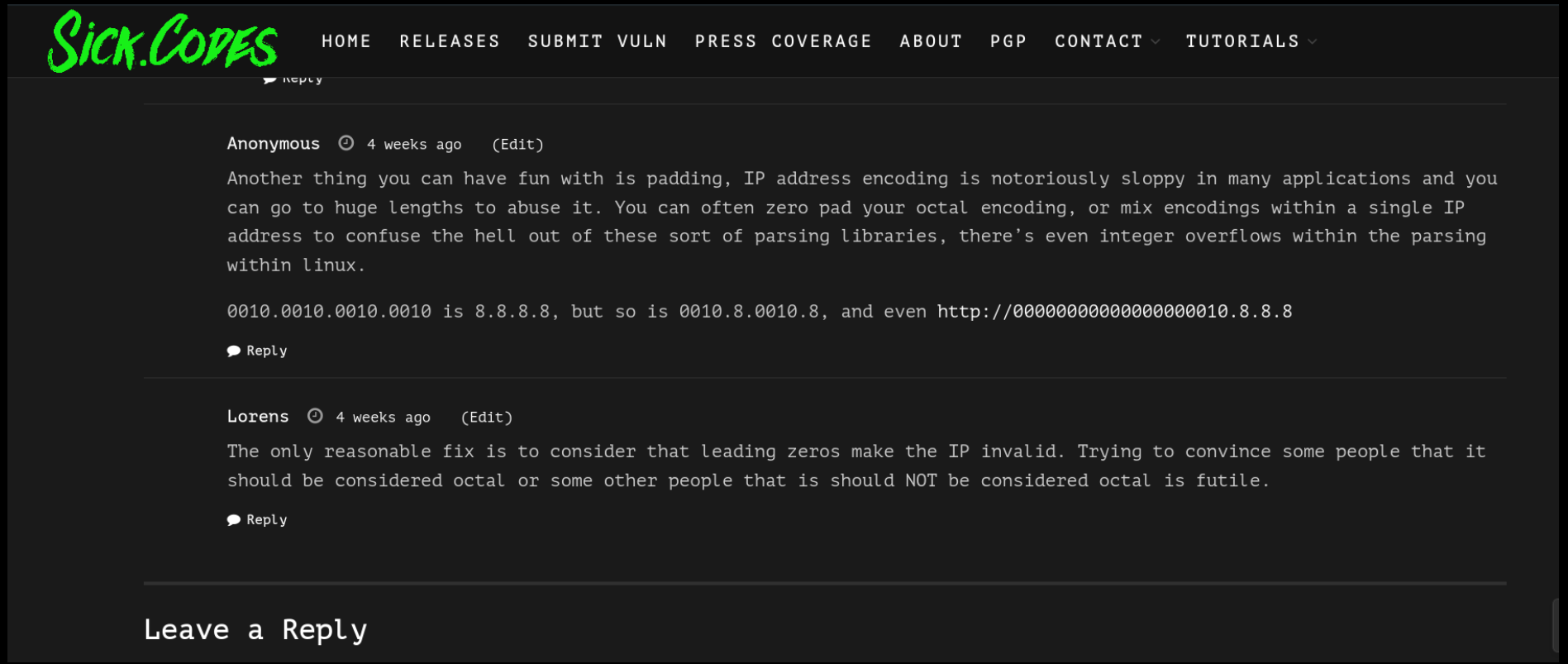
- **2,840,781** direct weekly downloads
- **170** publicly identified npm dependents
  (public npm packages relying on node-netmask)
- **289,515** GitHub dependents
  https://github.com/rs/node-netmask/network/dependents
- **238 million+** lifetime downloads
  (as of March 2021)

Net::Netmask

The Perl component Net::Netmask also suffered from this flaw (tracked under a separate identifier CVE-2021-29424), and its maintainer, Joelle Maslak has released a fix in the 2.0000 version today.

https://www.bleepingcomputer.com/news/security/critical-netmask-networking-bug-impacts-thousands-of-applications/

N-days the comments section



SICK.CODES HOME RELEASES SUBMIT VULN PRESS COVERAGE ABOUT PGP CONTACT TUTORIALS

Reply

Anonymous 4 weeks ago (Edit)

Another thing you can have fun with is padding, IP address encoding is notoriously sloppy in many applications and you can go to huge lengths to abuse it. You can often zero pad your octal encoding, or mix encodings within a single IP address to confuse the hell out of these sort of parsing libraries, there's even integer overflows within the parsing within linux.

0010.0010.0010.0010 is 8.8.8.8, but so is 0010.8.0010.8, and even http://00000000000000000010.8.8.8

Reply

Lorens 4 weeks ago (Edit)

The only reasonable fix is to consider that leading zeros make the IP invalid. Trying to convince some people that it should be considered octal or some other people that is should NOT be considered octal is futile.

Reply

Leave a Reply

Reply

Anonymous 4 weeks ago (Edit)

Another thing you can have fun with is padding, IP address encoding is notoriously sloppy in many applications and you can go to huge lengths to abuse it. You can often zero pad your octal encoding, or mix encodings within a single IP

Tony
ffchung

Maybe it only support octal format by design. Only the other people use it in wrong way ?

I just try java 8 and it only support octal format too.

Approve | Reply | Quick Edit | Edit | Spam | Trash

Universal "netmask" npm package, used by 270,000+ projects, vulnerable to octal input data: server-side request forgery, remote file inclusion, local file inclusion, and more (CVE-2021-28918)
View Post

5 4

2021/03/30 at 7:41 am

Reply

Leave a Reply

hackernews

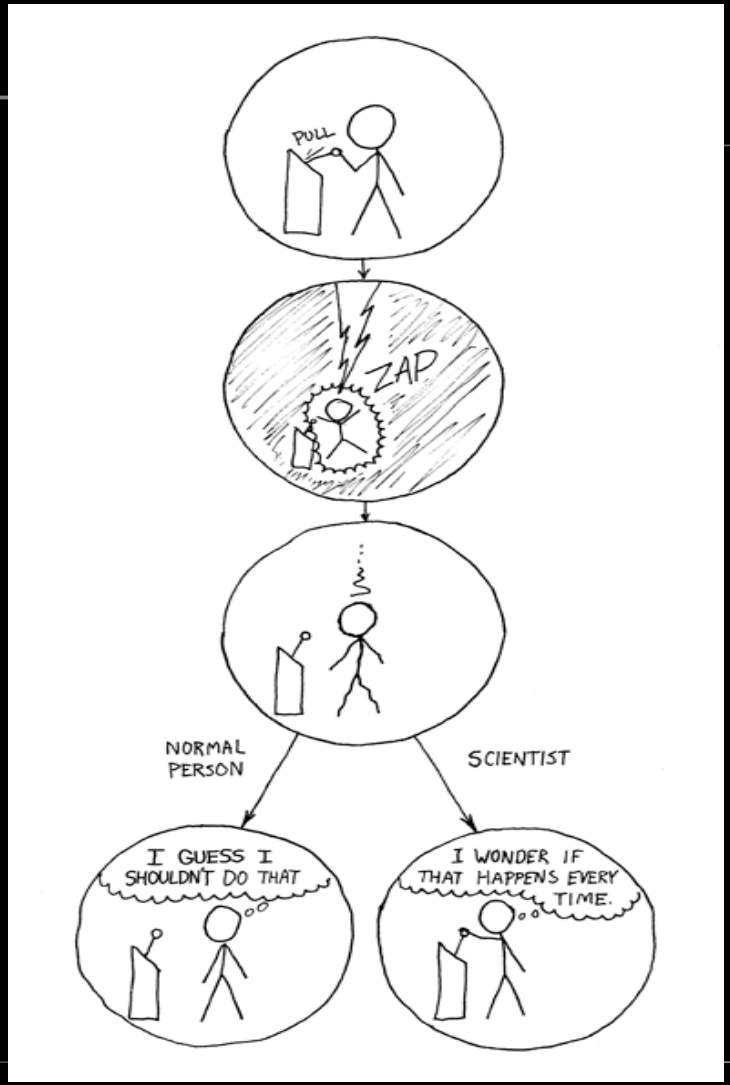
simion314 29 days ago [–]

Trust. I can trust code that is part of Java, .Net or Python standard library is good or decent quality. With npm you can't trust anything, package can go away, malware could be inserted in them, backward compatibility could brake.

Say you grab a 10 years old Java/.Net project source and a node one, and try to setup a developer environment to be able to create a fix. Would you bet that on average is 10X harder to get stuff to run on node ?

https://news.ycombinator.com/item?id=26620538

o rly



https://xkcd.com/242/

Applying the suggestion box

- Perl Data::Validate::IP, with Dave Rolsky (**CVE-2021-29662**)
- Python (**CVE-2021-29921**)
- ...

Meanwhile: discussing parseInt with Google

node (V8) returns:
```
4
5
6
7
8
9
8
```
However, it should absolutely be:
```
4
5
6
7
undef
undef
8
```

Meanwhile: discussing parseInt with Google

node (V8) returns:
```
Comment 3 by marja@chromium.org on Tue, Mar 30, 2021, 6:59 AM UTC (28 days ago) Project Member

**Status:** WontFix (was: Assigned)
**Owner:** marja@chromium.org
**Cc:** verwa...@chromium.org syg@chromium.org
**Components:** Blink>JavaScript>Language

Works as intended: 08 and 09 are allowed in non-strict mode as specced.

The spec:

https://tc39.es/ecma262/#sec-additional-syntax-numeric-literals
```

```
4
5
6
7
undef
undef
8
```

Meanwhile: discussing parseInt with Google

node (V8) returns:

```
Comment 3 by marja@chromium.org on Tue, Mar 30, 2021, 6:59 AM UTC (28 days ago) Project Member

Status: WontFix (was: Assigned)
Owner: marja@chromium.org
Cc: verwa...@chromium.org syg@chromium.org
Components: Blink>JavaScript>Language

Works as intended: 08 and 09 are allowed in non-strict mode as specced.

Our code:

// '\8' and '\9' are disallowed in strict mode.

https://source.chromium.org/chromium/chromium/src/+/master:v8/src/parsing/scanner.cc;l=414?ss=chromium

-> This is not a bug. Even if it was a bug, it wouldn't be a V8 security bug: random deviations from the spec (intentional
or unintentional) are not V8 security bugs per se. In particular, sometimes we need to deviate from the spec for web
compatibility.

However, in the case described in the bug we do follow the spec.
```

Meanwhile: discussing parseInt with Google

node (V8) returns:
```

Comment 3 by marja@chromium.org on Tue, Mar 30, 2021, 6:59 AM UTC (28 days ago) Project Member

**Status:** WontFix (was: Assigned)
**Owner:** marja@chromium.org
**Cc:** verwa...@chromium.org syg@chromium.org
**Components:** Blink>JavaScript>L

Works as intended: 08 and 09 a

Our code:

// '\8' and '\9' are disallowed

https://source.chromium.org/chromium

-> This is not a bug. Even if it
or unintentional) are not V8 sec
compatibility.

However, in the case described i
```

Meanwhile: discussing parseInt with Google

node (V8) returns:
```

Comment 3 by marja@chromium.org on Tue, Mar 30, 2021, 6:59 AM UTC (28 days ago) Project Member

**Status:** WontFix (was: Assigned)

> "So just confirming, downstream apps should really just be using 'use strict';, as introduced in ECMAScript 5?"

Generally yes, but I'm not sure if it helps with the bugs you're seeing down the line. "use strict" doesn't change the behavior or parseInt. It does change the behavior of octal literals. But I don't think anybody implements URL parsing in terms of octal literals.

To fix the original security bugs you were seeing, the solution is to implement URL parsing consistently as defined in the URL spec.

Comment 14 by info@sick.codes on Fri, Apr 2, 2021, 12:24 PM UTC (16 days ago)

Thanks for getting back to me in detail, and I appreciate the further explanation.
```

MDN's JavaScript error reference!

What went wrong?

Octal literals and octal escape sequences are deprecated and will throw a `SyntaxError` in strict mode. With ECMAScript 2015 and later, the standardized syntax uses a leading zero followed by a lowercase or uppercase Latin letter "O" (`0o` or `00`).

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Deprecated_octal

What went wrong?

Octal literals and octal escape sequences are deprecated and will throw a `SyntaxError` in strict mode. With ECMAScript 2015 and later, the standardized syntax uses a leading zero followed by a lowercase or uppercase Latin letter "O" (`0o` or `0O`).

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Deprecated_octal

What went wrong?

Decimal literals can start with a zero (`0`) followed by another decimal digit, but If all digits after the leading `0` are smaller than 8, the number is interpreted as an octal number. Because this is not the case with `08` and `09`, JavaScript warns about it.

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Bad_octal

Meanwhile: discussing parseInt with nodejs

3 #1141623 **Unexpected input validation of octal literals in nodejs v15.12.0 an d below returns defined values for all undefined octal literals.**

State	N/A (Closed)	Severity	Critical (10.0)
Disclosed	June 14, 2021 6:46am -0600	Participants	
Reported to	Node.js	Visibility	Disclosed (Full)
Reported at	March 30, 2021 8:26am -0600		
Asset	https://github.com/nodejs/node (Source code)		
CVE ID			
Weakness	Use of Inherently Dangerous Function		

Meanwhile: discussing parseInt with nodejs

3

#1141623

Unexpected input validation of octal literals in nodejs v15.12.0 an d below returns defined values for all undefined octal literals.

Supporting Material/References:

This bug I previously submitted to Chromium V8 (yesterday) which was rejected as "per spec"

However, this does not account for the fact that this is extremely dangerous for nodejs webapps, if not all nodejs web applications.

Mozilla interprets ECMA-262 octal literals containing 8 or 9 as not legal.

Code 85 Bytes Wrap lines Copy Download

```
1  08 is not a legal ECMA-262 octal constant.
2  09 is not a legal ECMA-262 octal constant.
```

The spec:

https://tc39.es/ecma262/#sec-additional-syntax-numeric-literals

Meanwhile: discussing parseInt with nodejs

3 #1141623 **Unexpected input validation of octal literals in nodejs v15.12.0 an**

mcollina Node.js staff posted a comment. Apr 1st (3 months ago)

The lines of code you are referring to are from V8 - Node.js vendors its dependencies.

I do not feel I am qualified to discuss spec compliance of JavaScript runtimes. I would recommend you to open an issue at https://github.com/tc39/ecma262 or https://bugs.chromium.org/p/v8/issues/list or anyway discuss this publicly with spec experts.

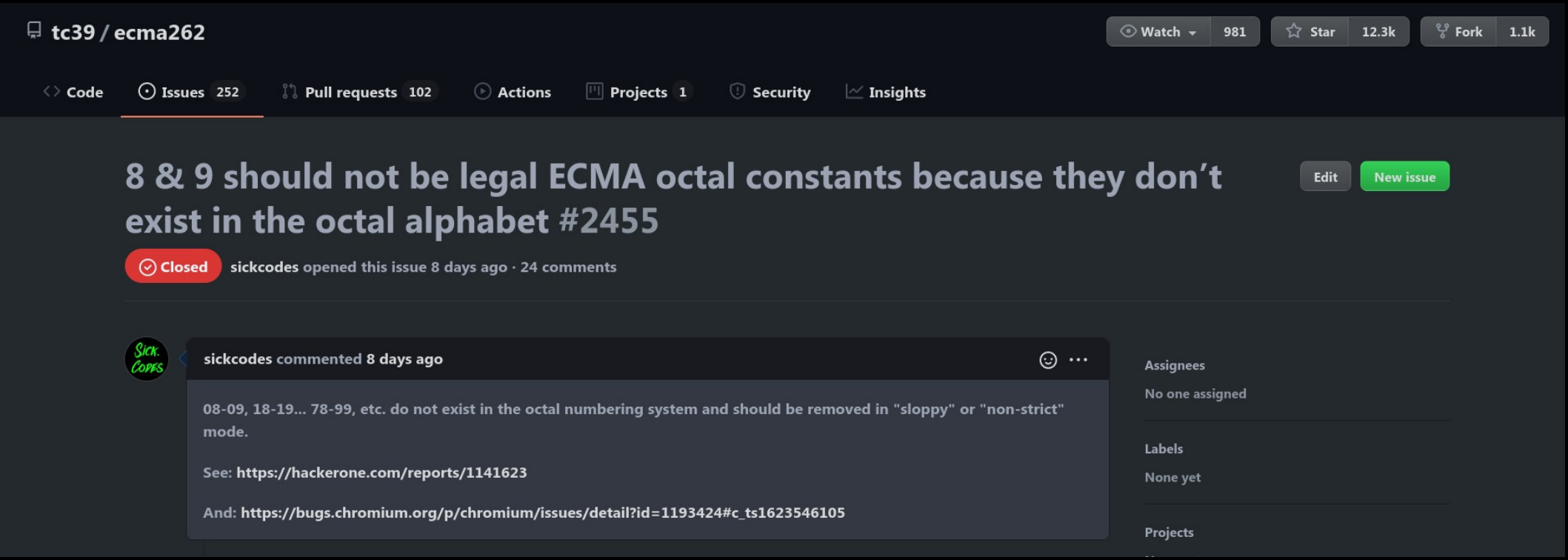
sickcodes closed the report and changed the status to ● **Not Applicable**. Apr 1st (3 months ago)

Totally understand, will open a public issue on there. Thanks very much for the discussion on this anyway, and for being super responsive!

mcollina Node.js staff posted a comment. Apr 1st (3 months ago)

it was really interesting, thanks for reporting!

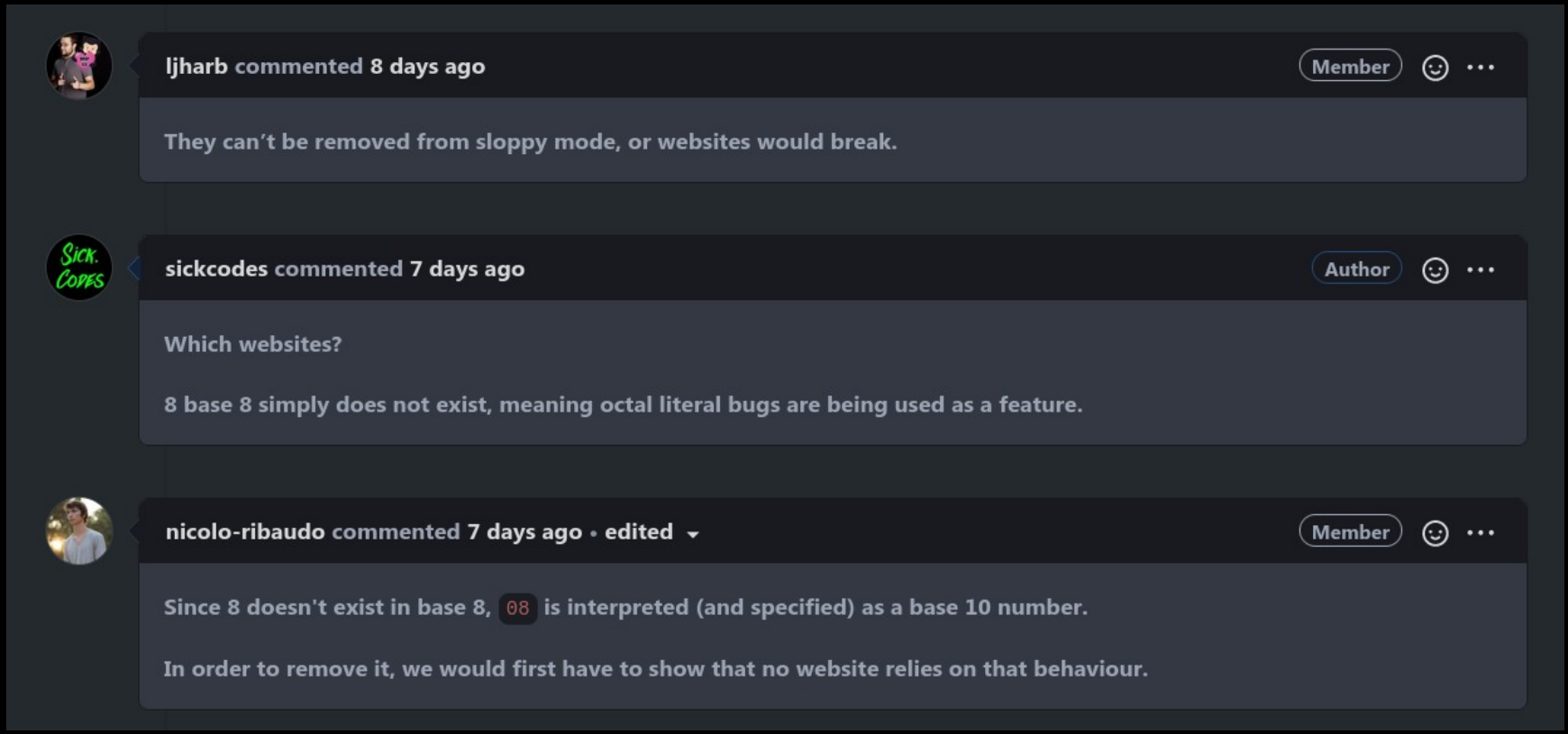
ECMA . . .



https://github.com/tc39/ecma262/issues/2455

@sickcodes

ECMA . . .



ljharb commented **8 days ago** — Member

They can't be removed from sloppy mode, or websites would break.

sickcodes commented **7 days ago** — Author

Which websites?

8 base 8 simply does not exist, meaning octal literal bugs are being used as a feature.

nicolo-ribaudo commented **7 days ago** • **edited** — Member

Since 8 doesn't exist in base 8, `08` is interpreted (and specified) as a base 10 number.

In order to remove it, we would first have to show that no website relies on that behaviour.

https://github.com/tc39/ecma262/issues/2455

sickcodes commented **5 days ago** — Author

> Lots of code relies on `018` being `18` too, almost certainly.
>
> At the very least, it is certainly the case that lots of code relies on `018` being *a number*, we can't make it an error and we can't make it `undefined`.

Okay but how can this be considered reliable:

`018` -> `18`
`020` -> `16`

Not sure how this can be considered reliable, so I assume code that relies on:

> `018` being *a number*

is unaware that it is flip flopping between base 8 and base 10.

bakkot commented **5 days ago** — Contributor

> Okay but how can this be considered reliable:

No one has claimed that this behavior makes sense, just that we can't change it now without breaking a bunch of sites, which we aren't going to do.

👍 1

Which Spiderman is responsible here?

Samuel King Jr. https://www.flickr.com/people/49840571@N02/, https://creativecommons.org/licenses/by-nc/2.0/

Some more PoCs

- CVE-2021-29922
- CVE-2021-29923
- Oracle S1446698*

We're also still tracking a few more we wish we could have shown.
Maybe next time

Java & the runtime environment

- javac
- Different JVMs
- Underlying C libraries!?

Few handle octal IPv4 addresses the same way.

Few handle ~~octal~~ IPv4 addresses the same way.

What should an IPv4 address actually be?

proactivesvcs 6 months ago [–]

I wonder how many of these bugs are the result of people thinking "Well I've read the spec but most of it is 'cursed' so I'll just implement this subset which fits my idea of 'acceptable'".

https://news.ycombinator.com/item?id=25545967

What should an IPv4 address actually be?

proactivesvcs 6 months ago [–]

I wonder how many of these bugs are the result of people thinking "Well I've read the spec but most of it is 'cursed' so I'll just implement this subset which fits my idea of 'acceptable'".

https://news.ycombinator.com/item?id=25545967

> Fun fact, the textual representation of IPv4 was never standardized in any document before IPv6 needed a grammar for its weirdo “trailing dotted quad” notation. So, it’s a de-facto standard that

https://blog.dave.tf/post/ip-addr-parsing/

What should an IPv4 address actually be?

Main expires 2005-08-23 [Page 1]

Internet-Draft Textual Representation of IP Addresses 2005-02-23

- Discussion of the ABNF grammar for IPv4 addresses given in [MTP] and [SMTP-1].
- Explanation of the faulty ABNF grammar in [IPV6-AA-2].
- Mention of another faulty grammar for IPv6 addresses in [SMTP-2].
- Specific references for the enclosing of IP addresses in brackets.
- Changed intended category from Informational to Standards-Track.

What should an IPv4 address actually be?

proactivesvcs 6 months ago [–]

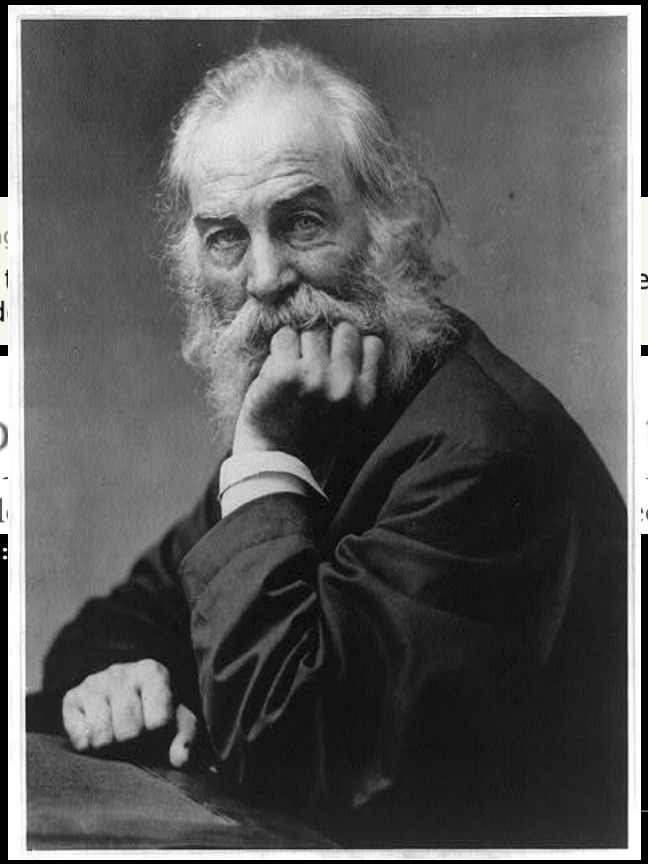
I wonder how many of these bugs are the result of people thinking "Well I've read the spec but most of it is 'cursed' so I'll just implement this subset which fits my idea of 'acceptable'".

If you want to *truly* parse IP addresses, this is the bullshit you have to put up with.

needed a grammar for its weirdo “trailing dotted quad” notation. So, it’s a de-facto standard that

https://blog.dave.tf/post/ip-addr-parsing/

What should an IPv4 address actually be?



proactivesvcs 6 months ag

I wonder how many of t
subset which fits my id

ell I've read the spec but most of it is 'cursed' so I'll just implement this

If you want to

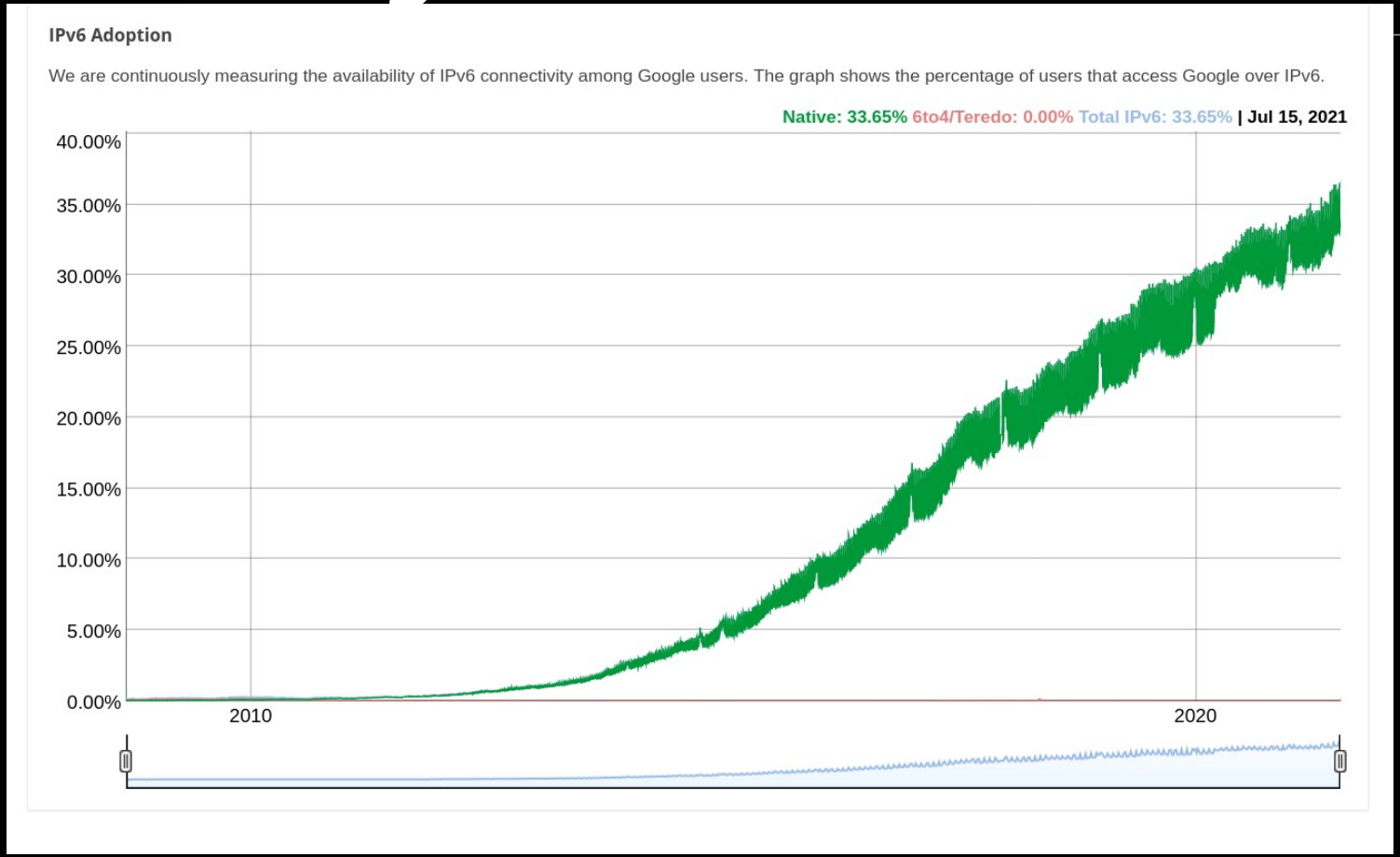
this is the bullshit you have to put up with.

need

d quad" notation. So, it's a de-facto standard that

https:

IPv6 though?



https://www.google.com/intl/en/ipv6/statistics.html

RFC draft 6943

[Search] [txt|html|pdf|bibtex] [Tracker] [WG] [Email] [Diff1] [Diff2] [Nits]

From: draft-iab-identifier-comparison-09 Informational
Internet Architecture Board (IAB) D. Thaler, Ed.
Request for Comments: 6943 Microsoft
Category: Informational May 2013
ISSN: 2070-1721

Issues in Identifier Comparison for Security Purposes

Abstract

Identifiers such as hostnames, URIs, IP addresses, and email addresses are often used in security contexts to identify security principals and resources. In such contexts, an identifier presented via some protocol is often compared using some policy to make security decisions such as whether the security principal may access the resource, what level of authentication or encryption is required, etc. If the parties involved in a security decision use different algorithms to compare identifiers, then failure scenarios ranging from denial of service to elevation of privilege can result. This document provides a discussion of these issues that designers should consider when defining identifiers and protocols, and when constructing architectures that use multiple protocols.

RFC draft 6943

[Search] [txt|html|pdf|bibtex] [Tracker] [WG] [Email] [Diff1] [Diff2] [Nits]

From: draft-iab-identifier-comparison-09 Informational
Internet Architecture Board (IAB) D. Thaler, Ed.
Request for Comments: 6943 Microsoft
Category: Informational May 2013
ISSN: 2070-1721

Issues in Identifier Comparison for Security Purposes

	"Grant on match"	"Deny on match"
False positive	Elevation of privilege	Denial of service
False negative	Denial of service	Elevation of privilege

algorithms to compare identifiers, then failure scenarios ranging from denial of service to elevation of privilege can result. This document provides a discussion of these issues that designers should consider when defining identifiers and protocols, and when constructing architectures that use multiple protocols.

RFC draft 6943

[Search] [txt|html|pdf|bibtex] [Tracker] [WG] [Email] [Diff1] [Diff2] [Nits]

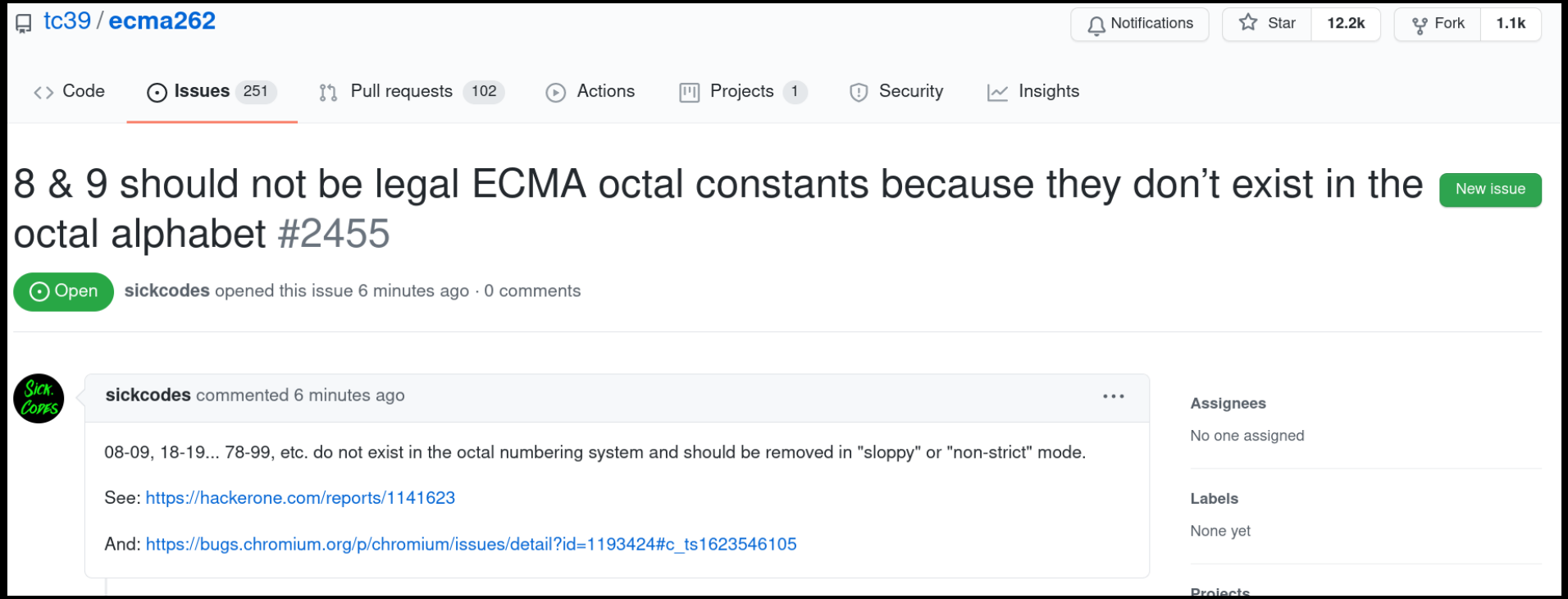
From: draft-iab-identifier-comparison-09 Informational
Internet Architecture Board (IAB) D. Thaler, Ed.
Request for Comments: 6943 Microsoft
Category: Informational May 2013
ISSN: 2070-1721

Issues in Identifier Comparison for Security Purposes

	"Grant on match"	"Deny on match"
False positive	Elevation of privilege	Denial of service
False negative	Denial of service	Elevation of privilege

algorithms to compare identifiers, then failure scenarios ranging from denial of service to elevation of privilege can result. This document provides a discussion of these issues that designers should consider when defining identifiers and protocols, and when constructing architectures that use multiple protocols.

Back to the original problem



Conclusion

- Teamwork
- Bug and vuln disclosure programs!
- Specification… maybe
- Don’t assume dependencies are perfect (please)

Thanks!